

Data Structures

Dilip Maripuri
Associate Professor
Department of Computer Applications



Collision Handling

Separate Chaining



- Separate chaining is a widely-used collision resolution strategy in hash tables that utilizes linked lists to handle collisions efficiently.
- At its core, separate chaining resolves collisions by allowing each position in the hash table, typically referred to as a "slot" or "bucket," to hold multiple elements.
- These elements are stored in a linked list attached to the respective slot.
- When a collision occurs—that is, when a newly inserted key hashes to a slot already occupied by one or more keys—the new key is simply added to the end of the list in that slot.



Data Structures

Collision Resolution – Separate Chaining



- **Key Features**
 - **Simple Implementation**
 - Separate chaining is straightforward to implement.
 - Each slot in the hash table starts as an empty list, and elements are added to these lists without the need for complex rehashing procedures.
 - **Dynamic Expansion**
 - Linked lists can grow as needed.
 - There is no need to specify an initial size, which makes separate chaining particularly effective in applications where the dataset size is unpredictable.



- **Steady Performance**

- Compiled by Dilip Maripuri

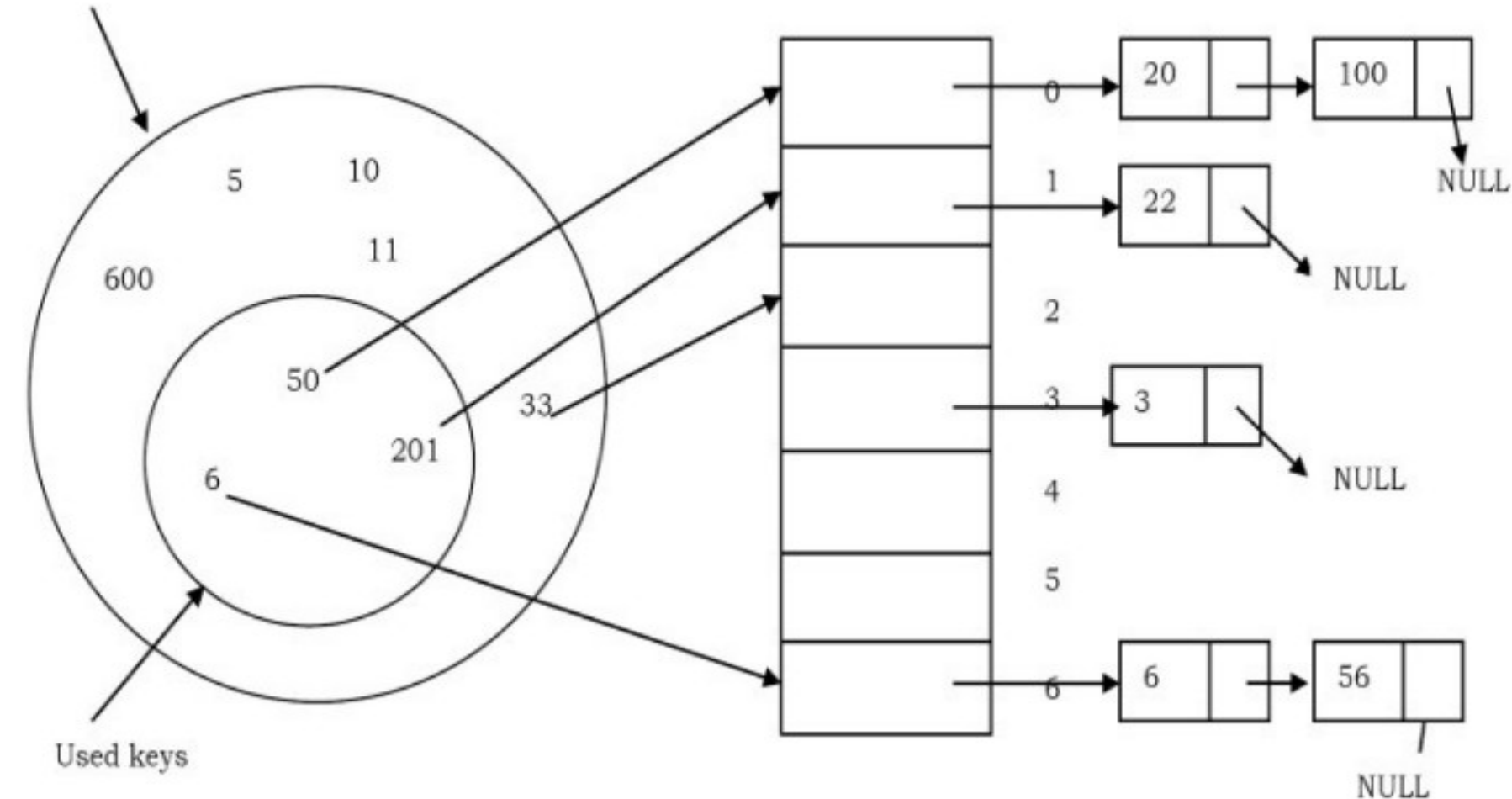


Data Structures

Collision Resolution – Separate Chaining



Universe of possible keys





- **Structure of the Hash Table**

- In a hash table implementing separate chaining, each slot in the array typically contains the hash key and a reference which is NULL initially.
- These slots will eventually reference the head nodes of linked lists.
- Each node of the list contains two parts:
 - Value: The data associated with the key.
 - Next: A pointer/reference to the next node in the list.



Data Structures

Collision Resolution – Separate Chaining



- **Inserting Elements**

- When inserting a new key-value pair, the hash table performs these steps:
 - The hash function calculates the index for the key.
 - If the slot at this index is empty (null), a new linked list is created, and the key-value pair forms the first node of this list.
 - If the slot is not empty, indicating a collision (another key has already been hashed to this index), the new node containing the value is added to the head or tail of the existing list at that index.
 - This choice can depend on the specific implementation details, such as ease of insertion or retrieval priorities.



Collision Resolution – Separate Chaining



- **Searching for Elements**

- To retrieve the value associated with a specific key:
 - The hash table computes the index for the key using the same hash function.
 - It then traverses the linked list at that index, comparing each node's key with the target key.
 - If a node with the matching key is found, its value is returned.
 - If the list is exhausted without finding the key, the search returns null or a failure indicator.



- **Removing Elements**

- Removing an element involves:
 - Calculating the index for the key
 - Traversing the linked list at that index to find and remove the node with the matching key.
 - If the node to be removed is found at the head of the list, the head pointer is adjusted to point to the next node.
 - If it's in the middle or end of the list, the previous node's next pointer is adjusted to skip the removed node, effectively unlinking it.



Data Structures

Collision Resolution – Separate Chaining



- **Load Factor**
 - The load factor of a hash table is the ratio of the number of elements to the number of slots.
 - A higher load factor means more collisions and longer chains, which can degrade performance.



- **Rehashing**

- To maintain efficient operations, separate chaining hash tables may implement rehashing, which involves creating a larger hash table and re-inserting all elements when the load factor crosses a certain threshold.
- This helps to distribute the elements more evenly and keep the chains shorter.



THANK YOU

Dilip Maripuri
Associate Professor
Department of Computer Applications
dilip.maripuri@pes.edu